



ИИ как ассистент программиста

МАСТЕР-КЛАСС

VS Code + Roo Code

Qwen3-Coder-Next

Qwen3.6

Local LLM

Что сегодня разберём



Обзор ИИ-ассистентов для разработки ПО



VS Code + Roo Code: настройка и режимы



Плюсы, минусы и лучшие практики использования ИИ-агентов · Вопросы



LLM-модели для агентов: обзор и характеристики



Демонстрация: решаем задачу с ИИ-агентом

Варианты ИИ-ассистентов для разработки ПО



Автодополнение

GitHub Copilot, Tabnine, Codeium —
строчное / блочное предсказание кода



Чат-ассистент

ChatGPT, Claude, Gemini — объяснение,
рефакторинг, генерация через диалог



ИИ-агент (Agent)

Roo Code, Cursor, Windsurf —
автономное многошаговое
выполнение задач



IDE-интеграция

Continue.dev, Amazon Q — глубокая
встройка в JetBrains, VS Code



Локальные модели

Ollama + Qwen / DeepSeek —
приватно, оффлайн, без расходов на
API



CI/CD + DevOps

Автоматизация тестов, ревью PR,
генерация документации в пайплайне

Сравнение инструментов

Топ ИИ-ассистентов 2025–2026

Инструмент	Тип	Цена	Особенность
GitHub Copilot	Автодополн.	\$10/мес	97% точность, 1.8M+ юзеров, GitHub интеграция
Cursor	IDE-агент	\$20/мес	VSCode-форк, лучший для сложного рефакторинга (88%)
Windsurf	IDE-агент	Free tier	Лучший бесплатный вариант, хорошая многофайловая работа
🌟 Roo Code	Open source	Бесплатно	5 режимов, локальные LLM, MCP, Boomerang
Claude Code	Терминал	API	Лучший для отладки, сложные задачи (91% рефакторинг)
Continue.dev	Open source	Бесплатно	VS Code + JetBrains, BYOM, гибкая настройка

· LLM для агентов

Какие LLM работают в роли агентов

Облачные

Claude 4 Sonnet/Opus — SWE-Bench
79.6%

GPT-4.1 / o3 — сильный reasoning

Gemini 2.5 Pro — 1M ctx, мультимод.

Зависимость от API Стоимость

Open-weight

DeepSeek V3 — сильный, MoE

Kimi K2 — лидер open-source коддинг

Mistral Large — Europe-friendly

Гибкость Self-host

Локальные (сегодня!)

Qwen3-Coder-Next — 70.6% SWE-Bench

Qwen3.6 35B-A3B — 73.4% SWE-Bench

DeepSeek-Coder-V2 — хорошая
альтернатива

Приватность Оффлайн

* SWE-Bench Verified — стандартный бенчмарк решения реальных GitHub-issues

Наши локальные модели

Qwen3-Coder-Next vs Qwen3.6



Qwen3-Coder-Next

80B-MoE · 3B активных · Apache 2.0

SWE-Bench: **70.6%** (Claude Sonnet 4.6 = 79.6%)Контекст: **256K** токеновСкорость: **60–120 tok/s** на H100GPU: **1× H100** или **2× RTX 5090**Фокус: **кодинг-задачи, агентика**

7.5T токенов обучения (70% код). Оптимизирован для agentic coding.



Qwen3.6 35B-A3B

35B-MoE · 3B активных · Apache 2.0

SWE-Bench: **73.4%** (превосходит Coder-Next!)AIME 2026: **92.6%**Контекст: **262K** (до 1M с YaRN)Архитектура: **Gated DeltaNet + MoE**Мультимодальн.: **текст + изображения**

Thinking Preservation: сохраняет контекст размышлений между запросами. Лучше для frontend и repo-level задач.



Стратегия: Qwen3-Coder-Next для Code/Debug режимов (быстрая генерация), Qwen3.6 для Architect/Orchestrator (сложное планирование)

• Основной инструмент

Roo Code — ИИ-агент прямо в VS Code

Open-source расширение · Поддержка 500+ моделей · Локальные LLM через Ollama · MCP-протокол



Code

Написание,
редактирование,
рефакторинг



Architect

Планирование систем и
архитектуры



Ask

Ответы и объяснение кода



Debug

Трассировка и
исправление багов



Orchestrator

Boomerang:
многоагентные задачи



Boomerang (Orchestrator mode): сложные задачи разбиваются на подзадачи, каждая уходит в нужный режим → результат возвращается. Избегает переполнения контекста и галлюцинаций.

Подключение локальных моделей через Ollama

1. Установка Ollama + модели

```
# Установить Ollama
curl -fsSL https://ollama.ai/install.sh | sh

# Скачать модели
ollama pull qwen3-coder-next
ollama pull qwen3.6

# Сервер запустится на
http://localhost:11434
```

Рекомендуемое распределение

Code mode → **qwen3-coder-next**
Debug mode → **qwen3-coder-next**
Architect mode → **qwen3.6**
Orchestrator → **qwen3.6**
Ask mode → **любая**

☒ Roo Code поддерживает назначение разных моделей на каждый режим — идеально для оптимизации баланса скорость/качество

2. Настройка в Roo Code (settings.json)

```
"provider": "openai-compatible",
"baseUrl": "http://localhost:11434/v1",
"model": "qwen3-coder-next"
```


· Демонстрация

Практическая задача:

■ CLI-инструмент для анализа логов

📄 Требования

- ✦ Читает nginx/app access.log
- ✦ Считает запросы по статус-кодам
- ✦ Топ-10 URL по запросам
- ✦ Топ-5 IP-адресов
- ✦ Статистика по времени ответа
- ✦ Вывод в таблицу + CSV-отчёт

🐍 Python 3 · argparse · re · csv · collections

🤖 Поток работы агента

1. **Architect mode (qwen3.6)**
→ Проектирует структуру модулей
2. **Code mode (qwen3-coder)**
→ Пишет parser.py, stats.py, cli.py
3. **Debug mode (qwen3-coder)**
→ Пишет тесты, правит ошибки

Быстрый путь от постановки до рабочего кода

Как формулировать задачу

✗ Плохой промпт:

Напиши скрипт для
анализа логов.

✓ Хороший промпт (Architect mode):

Спроектируй Python CLI-утилиту
для анализа `nginx access.log`.

Требования:

- Подсчёт статус-кодов (200, 404, 500)
- Топ-10 URL, топ-5 IP
- Среднее/макс время ответа
- Вывод: таблица + CSV файл

Предложи структуру модулей
и план реализации.

▴ Принципы хорошего промпта

- ✦ **Конкретная цель** — что должно делать
- ✦ **Технический стек** — язык, библиотеки
- ✦ **Входные данные** — формат, источник
- ✦ **Ожидаемый выход** — файлы, формат
- ✦ **Ограничения** — версии, зависимости

💡 После Architect → переключаем в **Code mode** с `qwen3-coder-next` для реализации

Что генерирует агент

log_analyzer/parser.py (фрагмент)

```
import re
from dataclasses import dataclass

LOG_PATTERN = re.compile(
    r'(?P<ip>\S+).*?\[(?P<time>[^\]]+)\]'
    r' "(?P<method>\S+) (?P<url>\S+).*?"'
    r' (?P<status>\d+) .* (?P<resp>\d+\.\d+)'
)

def parse_log(filepath: str):
    """Parse nginx access.log → entries"""
    with open(filepath) as f:
        for line in f:
            m = LOG_PATTERN.match(line)
            if m: yield m.groupdict()
```

Структура проекта

```
log_analyzer/
├─ parser.py ← regex парсинг
├─ stats.py ← Counter, агрегация
├─ reporter.py ← таблица + CSV
├─ cli.py ← argparse entry
└─ tests/
    ├─ test_parser.py
    └─ test_stats.py
```

✓ Агент сам создал файлы, написал тесты, исправил ошибки импорта без участия разработчика

Как работает ИИ-агент

Цикл Observe → Reason → Act



Observe

Читает файлы проекта,
структуру директорий,
контекст задачи



Reason

Планирует шаги, выбирает
инструменты, решает что
делать дальше



Act

Создаёт/редактирует файлы,
запускает терминал,
вызывает инструменты



Evaluate

Проверяет результат,
итерирует до выполнения
цели

🔧 Инструменты Roo Code

read_file · write_file · list_dir
execute_command (terminal)
browser_action · search_files
use_mcp_tool (расширяемо)

⚠️ Контроль действий

Auto-approve: чтение файлов
Confirm: запись, выполнение команд
Manual: деструктивные операции
Человек всегда в петле (HITL)

Преимущества и недостатки ИИ-агентов

✓ Преимущества

⚡ Скорость разработки

70% разработчиков отмечают снижение времени на рутинные задачи; 41% кода уже AI-assisted

🔧 Автоматические тесты

Генерация unit/integration-тестов, поиск edge cases без ручного труда

🔄 Рефакторинг и документация

Cursor: 88% на complex refactor vs 72% у Copilot. Автогенерация docstrings, README

🔒 Локальность и приватность

Код не покидает машину. Снижение стоимости на 60–80% vs чистый API

⚠ Недостатки и риски

🎭 Галлюцинации

Агент может уверенно сгенерировать несуществующие API, неверную логику

📄 Потеря контекста

На больших репозиториях контекстное окно не вмещает весь код. Нужен Boomerang

📉 Качество сложного кода

Слабее на нестандартные паттерны, глубокое TypeScript narrowing, novel алгоритмы

⚙ Требования к железу

Локальный запуск 35–80B моделей: нужен H100 или 2× RTX 5090 для оптимальной скорости

Как работать с агентом эффективно



Дробите задачи

Начинайте с простых bounded задач. Расширяйте функционал итеративно по мере проверки качества.



Human in the Loop

Всегда проверяйте изменения в diff. Агент предлагает — человек утверждает. Критически читайте код.



Контекст — это всё

Используйте `.clinerules` или `CLAUDE.md` с описанием архитектуры. Чем лучше контекст — тем лучше код.



TDD с агентом

Просите агент сначала написать тесты, потом реализацию. Это снижает число галлюцинаций на 30–40%.



Режим под задачу

Архитектура → Architect, код → Code, баги → Debug. Не используйте один режим на всё.



Измеряйте результат

Трекайте время на задачи с и без агента. Оценивайте качество кода, покрытие тестами, число ревизий.



Начните с малого

ИИ-агент — это мультипликатор вашей экспертизы, а не её замена. Используйте его как Junior-разработчика под вашим руководством.

[Roo Code](#)[Qwen3-Coder-Next](#)[Qwen3.6](#)[Ollama](#)[VS Code](#)

```
$ ollama pull qwen3-coder-next && code .
```